

Embedded Systems Course Project #1 - Spring 2026

Dead Lines:

11th Week (18 -23 April) initial assessment

13th Week (2 -7 May) Final Delivery

1. Project Title

Isolated-Word Speech Recognition on ATmega32A Using ADC-Based Audio Input

2. Project Overview

This project aims to design and implement a **real-time embedded speech recognition system** using the AVR ATmega32A microcontroller. The system will recognize **8 predefined spoken words** from an analog microphone input and activate one of **8 digital outputs** (e.g., LEDs) corresponding to the detected word. The project demonstrates how signal acquisition, digital signal processing, feature extraction, and lightweight classification can be implemented on a resource-constrained 8-bit microcontroller.

3. Learning Objectives

By completing this project, students will:

- Understand audio signal acquisition using ADC.
- Implement digital signal processing (DSP) on embedded hardware.
- Perform feature extraction (e.g., energy, zero-crossing rate, MFCC-lite).
- Implement a lightweight classification algorithm.
- Optimize code for memory and CPU limitations.
- Integrate training (offline) and testing (real-time) systems.

4. System Architecture

Input Stage

- Microphone captures speech
- Analog signal conditioning (amplification + filtering)
- Signal sampled using ATmega32A ADC

Processing Stage

- Preprocessing (normalization, framing)
- Feature extraction
- Word classification

Output Stage

- 8 digital output pins. Each output corresponds to one recognized word

5. Hardware Requirements

Core Components

- **ATmega32A**
- Electret microphone
- Microphone preamplifier (LM358 or similar op-amp)
- Low-pass anti-aliasing filter (~3–4 kHz cutoff)
- 8 LEDs
- USB-to-Serial converter
- Breadboard or PCB

Optional

- External SRAM (32KB or 64KB SRAM)
- LCD for debugging
- Push button for recording trigger

6. Software Requirements

Embedded Development

- AVR-GCC or Atmel Studio or Codevision (recommended)
- AVRDUDE (for flashing)
- Serial monitor for debugging

Training & Signal Processing (Offline on PC)

- Python.
- MATLAB (optional alternative)
- Audio recording software (Audacity)

7. Speech Recognition Approach

Because ATmega32A has limited RAM (2 KB) and Flash (32 KB), the system will use a **lightweight isolated-word recognition approach**, not deep learning.

Selected Approach:

1. Offline feature extraction (on PC)
2. Store reference feature templates in Flash memory
3. Online feature extraction (simplified)
4. Template matching using:
 - Euclidean distance
 - OR
 - Dynamic Time Warping (DTW-lite)

8. Project Procedure

Phase 1: Word Selection

Students choose 8 simple words such as:

- ON – OFF – START – STOP – LEFT – RIGHT – UP – DOWN – OPEN - CLOSE

Phase 2: Data Collection (Training Stage – PC Side)

1. Record 20–30 samples per word.
2. Convert audio to:
 - Mono
 - 8 kHz sampling rate
 - 8-bit resolution (to match MCU constraints)

Phase 3: Feature Extraction (Offline)

Extract lightweight features such as:

- Short-time energy
- Zero Crossing Rate (ZCR)
- Spectral centroid
- Simplified MFCC (optional advanced)
- Frame-based averaging

Generate one average feature vector per word.

Store the resulting feature vectors in a header file:

```
const float word_templates[8][N_FEATURES];
```

Phase 4: Hardware Setup

1. Build microphone amplifier circuit.
2. Connect output to ATmega32A ADC channel.

3. Configure:
 - ADC (8-bit mode)
 - Timer interrupt for 8 kHz sampling
4. Verify waveform through serial plotting.

Phase 5: Real-Time Processing on ATmega32A

1. Wait for speech trigger (energy threshold).
2. Record fixed-length buffer (e.g., 1 second).
3. Perform:
 - Framing
 - Energy calculation
 - ZCR calculation
4. Generate feature vector.
5. Compute distance from each stored template.
6. Select minimum distance.
7. Activate corresponding output pin.

Phase 6: Testing and Evaluation

Evaluate:

- Recognition accuracy (% correct)
- False detection rate
- Processing latency
- Memory usage
- CPU utilization

Test under:

- Quiet environment
- Moderate background noise
- Different speakers

9. Expected Challenges

- Memory limitations (You may need to use external 32KB SRAM for example)
- Real-time timing constraints
- Noise sensitivity
- Speaker variability
- ADC precision limits

10. Deliverables

Students must submit a Google drive link containing the following:

1. Circuit schematic
2. Source code
3. Training dataset
4. Report including:
 - System architecture
 - Feature extraction method
 - Classification method
 - Accuracy results
 - Optimization techniques
5. Demonstration videos illustrating the methodology, source code, and operation.
6. Delivery of the physical project and demonstrations for real time operation.

11. Project Extensions (Bonus Ideas)

- Add noise filtering
- Add UART command interface
- Implement speaker-dependent mode
- Store multiple templates per word
- Add LCD display of detected word